

Workshop 2: Intro to pandas

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn

NHH Norwegian School of Economics

October 9, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

Contents

1	Exercise 1: Data cleaning	1
2	Exercise 2: Selecting subsets of data	2
3	Exercise 3: Labor market statistics for the US	2
4	Exercise 4: Daily stock-market returns	3
5	Exercise 5: Working with string data (advanced)	3

Exercise 1: Data cleaning

Before doing actual data analysis, we usually first need to clean the data. This might involve steps such as dealing with missing values and encoding categorical variables as integers. In this exercise, you will perform such steps based on the Titanic passenger data.

1. Load the Titanic data set in `titanic.csv` located in the `data/` folder.
2. Report the number of observations with missing Age, for example using `isna()`.
3. Compute the average age in the data set. Use the following approaches and compare your results:

1. Use pandas's `mean()` method.

Hint: Select the Age column first, then call its `mean()` method.

2. Convert the Age column to a NumPy array using `to_numpy()`. Experiment with NumPy's `np.mean()` and `np.nanmean()` to see if you obtain the same results.

4. Replace all missing ages with the mean age you computed above, rounded to the nearest integer. You can either identify the missing values using a boolean mask and assign the mean age to those entries, or use `fillna()`.

Convert this updated Age column to integer type using `astype()`.

Hint: Use `astype(int)`.

5. Generate a new column `Female` which takes on the value one if `Sex` is equal to "female" and zero if `Sex` is equal to "male". This is called an *indicator* or *dummy* variable, and is useful for numerical analysis or regression. Delete the original column `Sex`.

Hint: Create a new column using `df['new_column'] = values`, replacing `new_column` with the desired column name and `values` with the data to store.

6. Save your cleaned data set as `titanic-clean.csv` using `to_csv()` with `,` as the field separator. Tell `to_csv()` to *not* write the `DataFrame` index to the CSV file as it's not needed in this example.

Exercise 2: Selecting subsets of data

In this exercise, you are asked to select subsets of macroeconomic data for the United States based on some criteria.

1. Load the annual data from FRED which are located in `FRED_annual.xlsx` in the `data/FRED` folder. Print the list of columns and the number of non-missing observations.
Since we are dealing with time series data, set the column `Year` as the `DataFrame` index.
2. Print all observations for the 1960s decade using at least two different methods.
3. Using the data in the column `GDP`, compute the annual GDP growth in percent and store it in the column `GDP_growth`. Select the years in which

1. GDP growth was above 5%.
2. GDP growth was negative, but inflation was still above 5% (such episodes are called "stagflation" since usually negative GDP growth is associated with low inflation).

Use at least two methods for the stagflation selection.

Hint: You can compute changes relative to the previous observation using the `pct_change()` method.

Exercise 3: Labor market statistics for the US

In this exercise, you are asked to compute some descriptive statistics for the unemployment rate and the labor force participation rate for the United States. The latter is the percentage of the civilian noninstitutional population aged 16 and older who are in the labor force (employed or unemployed). Both rates are expressed as percentages.

1. Load the monthly time series from FRED which are located in `FRED_monthly.csv` in the `data/FRED` folder.

Hint: Use `pd.read_csv(..., parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates.

Since we are dealing with time series data, set the column `DATE` as the `DataFrame` index. Then print the list of columns and the number of non-missing observations.

2. For the columns `UNRATE` (unemployment rate) and `LFPART` (labor force participation rate), compute and report the mean, minimum, and maximum values for the whole sample. Round your results to one decimal place.

Hint: You can use the `DataFrame` methods `mean()`, `min()`, and `max()` to compute the desired statistics.

Hint: You can use the `DataFrame` method `round()` to round the results to one decimal place.

3. You are interested in how the average unemployment rate evolved over the last few decades.

- Add a new column `Decade` to the `DataFrame` which contains the starting year for each decade (e.g., this value should be 1950 for the years 1950-1959, and so on).

Hint: The decade can be computed from the column `Year` using truncated integer division:

```
df['Year'] // 10 * 10
```

- Write a loop to compute and report the average unemployment rate (column `UNRATE`) for each decade.

Include only the decades from 1950 to 2010 for which you have all observations.

Exercise 4: Daily stock-market returns

In this exercise, you will compute daily returns for the S&P 500 index and count trading days with losses or large gains. The column `SP500` contains the index values.

1. Load `SP500.csv` from the `data/stockmarket` folder, parse `Date` as dates, and set it as the `DataFrame` index. Restrict the data to observations after January 1, 2020.

Hint: Use `pd.read_csv(..., parse_dates=['Date'])` to automatically parse the strings in the `Date` column as dates.

2. Compute daily returns in percent using `pct_change()` and store them in a new column `Return`. Explain why the first return is missing.

Hint: Multiply the result of `pct_change()` by 100 to express returns in percent.

3. Report the number and percentage of trading days with losses, i.e., negative returns. Exclude days with missing returns from the denominator.

Hint: Summing a Boolean Series counts the number of True values. Use `count()` to count nonmissing observations.

4. Report the number and percentage of trading days with gains of more than 5%.

5. Report the number and percentage of days with negative returns on both the current and the previous trading day. A run of three loss days contributes two qualifying days. For the percentage, include only days with both returns observed.

Hint: Use `shift()` to obtain the previous trading day's return, and combine the two conditions using `&`.

Exercise 5: Working with string data (advanced)

Most of the data we deal with contain strings, i.e., text data (names, addresses, etc.). Often, such data is not in the format needed for analysis, and we have to perform additional string manipulation to extract the exact data we need. This can be achieved using the pandas [string methods](#).

To illustrate, we use the Titanic data set for this exercise.

1. Load the Titanic data and restrict the sample to men. (This simplifies the task. Some married women's entries contain both their husband's name and their maiden name.)
2. Print the first five observations of the `Name` column. As you can see, the data is stored in the format "*Last name, Title First name*" where the title is something like Mr., Rev., etc.
3. Split the `Name` column by `,` to extract the last name and the remainder as separate columns. You can achieve this using the `partition()` string method.

4. Split the remainder (containing the title and first name) using the space character " " as separator to obtain individual columns for the title and the first name.

Hint: The remainder starts with a space after the comma. Use `.str.strip()` to remove leading and trailing whitespace before splitting it.

5. Store the three data series in the original DataFrame (using the column names `FirstName`, `LastName`, and `Title`) and delete the `Name` column, which is no longer needed.
6. Finally, extract the ship deck from the values in `Cabin`. The ship deck is the first character in the string stored in `Cabin` (A, B, C, ...). You extract the first character using the `get()` string method. Store the result in the column `Deck`.

Hint: pandas's string methods can be accessed using the `.str` attribute. For example, to partition values in the column `Name`, you need to use

```
df['Name'].str.partition()
```